

# CS570: Big Data Processing & Analytics

---

## Week 2 Lab: HDFS & Distributed Thinking

**Instructor:** Dr. Ragnar Lesch

**Duration:** 60 minutes

**Date:** Week 2, Spring 2026

---

## Prerequisites

Before starting this lab, ensure you have:

- **Python 3.11** installed
  - **PySpark 4.1.x** installed (`pip install pyspark`)
  - **JupyterLab** installed and able to launch (`jupyter lab`)
  - **Hadoop 3.4.2** installed with HDFS running (Mac via Homebrew, Windows via WSL2)
  - Can run commands in terminal (Mac) or WSL2 (Windows)
- 

## Hadoop Installation (If Not Already Installed)

**Most students will need to install Hadoop before this lab.** Follow the instructions for your operating system:

### Mac (using Homebrew):

```
# Install Hadoop
brew install hadoop

# Configure Hadoop environment
echo 'export HADOOP_HOME=/opt/homebrew/opt/hadoop/libexec' >> ~/.zshrc
echo 'export PATH=$HADOOP_HOME/bin:$PATH' >> ~/.zshrc
echo 'export HADOOP_OPTS="$HADOOP_OPTS -Dhadoop.native.lib=false"' >>
~/.zshrc
source ~/.zshrc

# Verify installation
hadoop version
```

**Expected output:** `Hadoop 3.4.2` (or similar 3.4.x version on Mac)

**Note:** The third `export` line suppresses a harmless warning about native libraries on Mac.

---

### Windows (using WSL2 - Recommended):

## What is WSL2?

WSL2 (Windows Subsystem for Linux) gives you a **real Linux environment inside Windows**. You'll install Ubuntu Linux and run all Hadoop commands there - **NOT in PowerShell or CMD**.

Think of it as a lightweight Linux virtual machine built into Windows. Hadoop was designed for Linux and doesn't work properly in Windows PowerShell, so WSL2 lets you run it the way it was meant to run.

### After setup, you'll:

- Open the "Ubuntu" app from Windows Start Menu (not PowerShell!)
- Use Linux commands (**ls**, **cd**, **hdfs**) in the Ubuntu terminal
- Keep your Windows files separate from your Linux/Hadoop files

---

#### 1. Install WSL2 (if not already):

- Open PowerShell as Administrator
- Run: **wsl --install**
- Restart computer

#### 2. Install Ubuntu from Microsoft Store

#### 3. Inside Ubuntu WSL, install Hadoop:

```
# Update package list
sudo apt update

# Install Java (required for Hadoop)
sudo apt install openjdk-11-jdk -y

# Download Hadoop
wget https://dlcdn.apache.org/hadoop/common/hadoop-3.4.2/hadoop-3.4.2.tar.gz

# Extract
tar -xzvf hadoop-3.4.2.tar.gz
sudo mv hadoop-3.4.2 /usr/local/hadoop

# Set environment variables
echo 'export HADOOP_HOME=/usr/local/hadoop' >> ~/.bashrc
echo 'export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin' >> ~/.bashrc
echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64' >> ~/.bashrc
source ~/.bashrc

# Verify
hadoop version
```

**Expected output:** **Hadoop 3.4.2**

---

## Configure HDFS (All Platforms)

After installing Hadoop, you need to configure HDFS. **These commands are identical for Mac and Windows/WSL2:**

```
# Create core-site.xml configuration
cat > $HADOOP_HOME/etc/hadoop/core-site.xml << 'XMLEOF'
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
XMLEOF

# Create hdfs-site.xml configuration
cat > $HADOOP_HOME/etc/hadoop/hdfs-site.xml << 'XMLEOF'
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
XMLEOF
```

**What this does:**

- **core-site.xml**: Tells Hadoop to use HDFS at **localhost:9000**
- **hdfs-site.xml**: Sets replication to 1 (sufficient for single-machine setup)

---

## Set Up Passwordless SSH (All Platforms)

Hadoop requires SSH to start its daemons, even on a single machine. The SSH key setup is the same for Mac and Windows, but the initial setup differs slightly.

---

Mac: Enable Remote Login First

**Before generating SSH keys, enable Remote Login:**

1. Open **System Settings** (or System Preferences on older macOS)
2. Go to **General** → **Sharing** (or just **Sharing** on older macOS)
3. Turn on **Remote Login**
4. Make sure your user is in the "Allow access for" list

---

Windows/WSL2: Install SSH Service First

**Before generating SSH keys, install and start SSH:**

```
# Install SSH server
sudo apt install openssh-server -y

# Start SSH service
sudo service ssh start
```

---

**Generate SSH Keys (Same for Mac and Windows)****Now run these commands (identical on both platforms):**

```
# Generate SSH key
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa

# Add your key to authorized_keys
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys

# Test SSH to localhost (should connect without password)
ssh localhost
```

**First time connecting, you'll see:**

```
The authenticity of host 'localhost' can't be established.
Are you sure you want to continue connecting (yes/no)?
```

Type **yes** and press Enter, then type **exit** to logout.

**If this works without asking for a password, you're all set!**

---

**Start and Verify HDFS (All Platforms)**

**These steps are the same whether you're on Mac or Windows/WSL2.**

---

**Step 1: Format HDFS (First Time Only)**

```
# Format the HDFS namenode (only do this ONCE)
hdfs namenode -format
```

**Expected output:** Should end with "Storage directory ... has been successfully formatted."

**\*\* Important:\*\*** Only run this ONCE during initial setup. Running it again will erase all HDFS data!

---

## Step 2: Start HDFS Services

```
# Start HDFS services
start-dfs.sh
```

**You may see these messages (they're harmless):**

```
ERROR: Cannot set priority of namenode process
ERROR: Cannot set priority of datanode process
ERROR: Cannot set priority of secondarynamenode process
```

**Note:** These are just warnings on Mac - HDFS still starts correctly. On Windows/WSL2 you might not see these.

---

## Step 3: Verify HDFS is Running

```
# Check that HDFS processes are running
jps
```

**Expected output:**

```
12345 NameNode
12346 DataNode
12347 SecondaryNameNode
12348 Jps
```

**If you see all three processes (NameNode, DataNode, SecondaryNameNode), HDFS is running!**

**Note:** The numbers will be different - just verify the process names appear.

---

## Step 4: Test HDFS Commands

```
# Test HDFS is accessible
hdfs dfs -ls /
```

Should work without errors (may be empty on first run).

---

## Step 5: Check HDFS Web Interface (Optional)

**Open in your browser:** `http://localhost:9870`

You should see the NameNode web UI showing:

- Cluster overview
- DataNodes (should show 1 live node)
- File browser
- Configuration

**For Windows/WSL2:** Access from Windows browser (not inside WSL2).

---

## Installation Time Estimate

- **Mac:** 20-25 minutes (install + config + SSH + start)
- **Windows (WSL2):** 25-35 minutes (WSL setup + install + config + SSH + start)

**Do this BEFORE lab** to maximize hands-on time!

---

## Quick Troubleshooting

### "Command not found: hadoop"

- Check environment variables are set
- Run `source ~/.zshrc` (Mac) or `source ~/.bashrc` (WSL2)

### "NameNode won't start"

- Try reformatting: `hdfs namenode -format`
- Check logs: `$HADOOP_HOME/logs/`

### "Port 9000 already in use"

- Something else is using the port
- Change port in `core-site.xml` or stop conflicting service

### Still stuck?

- Come to office hours (Thursday 4-5 PM)
  - Post in course Slack channel
  - We'll help debug during lab if needed
- 

**Note:** This lab assumes you've successfully set up your environment. If you encounter issues, ask for help early—don't spend 30 minutes stuck on setup!

---

## Learning Objectives

By the end of this lab, you will:

1. Use HDFS command-line interface to interact with distributed storage
2. Understand how data is distributed across a cluster through blocks and replication
3. Apply MapReduce thinking to solve the max-temperature problem
4. See the connection between HDFS concepts and Spark operations
5. Experience the difference between Hadoop's disk-based and Spark's in-memory processing

---

## Lab Structure

This lab is completed **entirely in a JupyterLab notebook**:

- **Part 1 (20 min)**: HDFS CLI commands (run in notebook using `!` prefix)
- **Part 2 (40 min)**: PySpark coding
- **Part 3**: Reflection questions

All commands, code, outputs, and answers go in **one notebook**. At the end, you'll export it as a single PDF.

---

## Part 1: HDFS Command Line Interface

---

**Time:** 20 minutes

**Environment:** JupyterLab Notebook

You can run shell commands directly in Jupyter by prefixing them with `!`. The output gets captured in the notebook — no screenshots needed.

---

### Exercise 1.1: Setup Verification (3 minutes)

Create a new code cell and verify your environment:

```
# Check Hadoop installation
!hadoop version

# Check HDFS is running
!hdfs dfs -ls /

# Check PySpark
import pyspark
print(pyspark.__version__)
```

#### Expected Output:

- Hadoop version should show **Hadoop 3.4.2**
- HDFS listing should work without errors
- PySpark version should show **4.1.x** (e.g., **4.1.1** or newer)

**If any command fails:** Stop and troubleshoot before continuing. See troubleshooting section at the end.

---

## Exercise 1.2: Create and Upload Data to HDFS (5 minutes)

### Step 1: Create your HDFS workspace

```
# Create a directory for this week's work
!hdfs dfs -mkdir -p /user/${whoami}/week2
```

**What's happening?** The `-p` flag creates parent directories if they don't exist (just like `mkdir -p` in Linux). Without it, the command would fail if `/user/${whoami}` doesn't exist yet.

---

### Step 2: Create sample data locally

#### Option A: Small dataset (quick)

```
# Create a local text file with e-commerce transaction data
sample_data = """customer_001,laptop,1299.99,San Francisco,completed
customer_002,headphones,89.99,New York,completed
customer_001,mouse,24.99,San Francisco,completed
customer_003,keyboard,159.99,Los Angeles,pending
customer_002,monitor,449.99,New York,completed"""

with open("sample.txt", "w") as f:
    f.write(sample_data)

print("Created sample.txt")
```

**What's this?** E-commerce transaction data - customer ID, product, price, city, status. This is the kind of data companies like Amazon process at massive scale with HDFS.

#### Option B: Larger dataset (optional - more realistic)

If you want to see HDFS with more realistic data sizes, generate a larger file:

```
import random

cities = ["San Francisco", "New York", "Los Angeles", "Chicago", "Boston"]
statuses = ["completed", "pending", "cancelled"]

with open("large_transactions.csv", "w") as f:
    for i in range(100000):
        customer_id = f"customer_{random.randint(1, 500):03d}"
        product = f"product_{random.randint(1, 100)}"
        price = f"{random.randint(1, 1000)}.{random.randint(0, 99):02d}"
        city = random.choice(cities)
        status = random.choice(statuses)
        f.write(f"{customer_id},{product},{price},{city},{status}\n")
```



```
print("Generated large_transactions.csv with 100,000 rows")
```

**Note:** This takes ~5-10 seconds to generate. Use this if you want to see how HDFS handles larger files (though it's still under 128MB, so still only 1 block).

For the rest of the exercises, we'll assume you're using the small `sample.txt` file. If you used the large file, replace `sample.txt` with `large_transactions.csv` in all commands.

---

### Step 3: Upload to HDFS

```
# Upload file to HDFS
!hdfs dfs -put sample.txt /user/${whoami}/week2/

# Verify upload
!hdfs dfs -ls /user/${whoami}/week2/
```

#### Expected Output:

```
Found 1 items
-rw-r--r--    1 your_username supergroup      246 2026-01-XX 18:00
/user/your_username/week2/sample.txt
```

**Note the "1"** - that's the replication factor we configured!

---

### Step 4: Check file metadata

```
# Check file status and replication
!hdfs dfs -stat "%b %o %r %n" /user/${whoami}/week2/sample.txt
```

**Expected Output:** 246 134217728 1 sample.txt

- 246 = file size in bytes (5 e-commerce transactions)
- 134217728 = block size (128MB in bytes)
- 1 = replication factor (we set this to 1 for single-machine setup)
- sample.txt = filename

---

**Question 1:** What is the replication factor of your file? Why is it set to 1 instead of the default 3?

Create a markdown cell below and write your answer there.

---

## Exercise 1.3: Understanding Block Storage (5 minutes)

## Step 1: Examine block details

```
# Check block locations and health
!hdfs fsck /user/$(whoami)/week2/sample.txt -files -blocks -locations
```

### Expected Output (key parts to look for):

```
/user/leschrh69/week2/sample.txt 246 bytes, replicated: replication=1, 1
block(s): OK
0. BP-....blk_1073741825_1001 len=246 Live_repl=1
  [DatanodeInfoWithStorage[127.0.0.1:9866,DS-...,DISK]]

Status: HEALTHY
Number of data-nodes: 1
Number of racks: 1
Total blocks (validated): 1
Default replication factor: 1
Average block replication: 1.0
```

### What to notice:

- **1 block** (file is 246 bytes, under 128MB)
- **Live\_repl=1** (one live replica)
- **Only 1 DataNode** at **127.0.0.1:9866** (localhost - our single machine)
- **Number of racks: 1** (default rack on single machine)
- **Status: HEALTHY**

---

### Question 2: Answer these based on your **fsck** output:

- How many blocks does your file have?
- On which DataNode is the block stored? (IP address and port)
- How many racks does your cluster have?
- Why do you only see 1 DataNode instead of multiple DataNodes?

Create a markdown cell below and write your answers there.

---

## Exercise 1.4: Replication in Action (4 minutes)

### Step 1: Increase replication factor

```
# Change replication from 1 to 2
!hdfs dfs -setrep 2 /user/$(whoami)/week2/sample.txt
```

**What's happening?** We're setting the *desired* replication to 2. On a single machine with only 1 DataNode, HDFS can't actually create 2 copies - it will remain at 1 actual copy but show desired replication of 2. In a real multi-node cluster, HDFS would create 2 copies on different machines.

**Note:** We removed the `-w` (wait) flag because it would hang forever on a single-machine setup waiting for a second copy that can never be created.

---

## Step 2: Verify the change

```
# Check new replication factor
!hdfs dfs -stat "%r" /user/$(whoami)/week2/sample.txt

# See updated block locations
!hdfs fsck /user/$(whoami)/week2/sample.txt -files -blocks -locations
```

---

**Question 3:** What changed after setting replication to 2?

- a) What replication factor does `stat` show now?
- b) On a single-machine setup, how many actual copies exist? Why?
- c) What would happen on a 3-node cluster with replication set to 2?

Create a markdown cell below and write your answers there.

---

## Exercise 1.5: Data Locality Verification (3 minutes)

### Step 1: Download from HDFS

```
# Download file back to local system
!hdfs dfs -get /user/$(whoami)/week2/sample.txt downloaded_sample.txt
```

---

### Step 2: Compare files

```
# Compare original and downloaded
!diff sample.txt downloaded_sample.txt
```

**Expected Output:** *No output* (files are identical)

---

**Question 4:** Why does HDFS try to run Map tasks on the same machine that stores the data blocks?

Create a markdown cell below and write your answer there.

## Exercise 1.6: Under the Hood (Optional - 5 minutes)

### Part A: See Where HDFS Actually Stores Your Data

HDFS stores blocks as regular files on your local disk. Let's find them:

#### Step 1: Ask HDFS where it stores data

```
# Get the configured data directory
!hdfs getconf -confKey dfs.datanode.data.dir
```

This will show something like `file:///tmp/hadoop-{username}/dfs/data`

#### Step 2: Find your block files

```
import subprocess
import os

# Get the data directory path
result = subprocess.run(['hdfs', 'getconf', '-confKey',
                          'dfs.datanode.data.dir'],
                        capture_output=True, text=True)
data_dir = result.stdout.strip().replace('file://', '')
print(f>Data directory: {data_dir}")

# Find all block files
!find {data_dir} -name "blk_*" -type f 2>/dev/null | head -10
```

#### Step 3: Read a block file directly

```
# List block files with details
!ls -lh {data_dir}/current/BP-*/current/finalized/subdir0/subdir0/blk_*
2>/dev/null | head -5

# Read the actual block file - it contains your data!
!cat {data_dir}/current/BP-
*/current/finalized/subdir0/subdir0/blk_1073741*(!.meta) 2>/dev/null |
head
```

You should see actual block files like `blk_1073741826` (data) and `blk_1073741826_1002.meta` (checksums). **The block file is just a regular file containing your data!**

**Question 5 (Optional):** HDFS shows your file at `/user/{whoami}/week2/sample.txt`, but physically it's stored as block files in `/tmp/hadoop-.../dfs/data/...`. Why does HDFS use this abstraction instead of just using regular file paths? What benefits does it provide?

Create a markdown cell below and write your answer there.

---

## Part B: HDFS Data Persistence

⚠ **Run these commands in a separate terminal window, not in the notebook.** Stopping HDFS while the notebook is running can cause connection issues.

Understand how HDFS data persists on disk even when services stop:

```
# In a terminal window (not the notebook):

# Stop HDFS services
stop-dfs.sh

# HDFS commands won't work (no NameNode running)
hdfs dfs -ls /user/$(whoami)/week2/
# This will fail with "Connection refused"

# But the actual data files still exist on disk!
find /tmp/hadoop-$(whoami)/dfs/data -name "blk_*" 2>/dev/null | head -3
# You can still see the block files

# Restart HDFS
start-dfs.sh
jps # Verify NameNode and DataNode are running

# Now HDFS commands work again
hdfs dfs -ls /user/$(whoami)/week2/
# Your file is back!
```

### What this demonstrates:

- HDFS metadata (file locations) lives in NameNode memory
- Actual data blocks persist on DataNode disks
- When services restart, HDFS rebuilds its state from disk
- In a multi-node cluster, if 1 node stops, other replicas remain available

**Note:** This is NOT simulating a true node failure (you can't do that on a single machine). This shows data persistence and the separation between metadata (NameNode) and data (DataNodes).

**Question 6 (Optional):** In a 3-node cluster with replication=3:

- What happens if 1 DataNode stops? Can you still access your files?
- What happens if 2 DataNodes stop? Can you still access your files?
- What happens if the NameNode stops? What about the data?

*Hint: Think about the difference between metadata (file locations) and actual data (blocks).*

Create a markdown cell below and write your answers there.

---

# Part 2: Max Temperature Problem with PySpark

---

**Time:** 40 minutes

**Environment:** Continue in the same JupyterLab Notebook

---

## Getting Started

You're already in JupyterLab from Part 1. Continue in the same notebook — add a markdown cell with `## Part 2: PySpark` to separate the sections.

---

## Problem Description

You have weather data for multiple cities over several days. Each record contains:

- **City name**
- **Date**
- **Temperature (°F)**

**Goal:** Find the maximum temperature recorded for each city.

### MapReduce Connection:

- **Map:** Extract (city, temperature) pairs
  - **Shuffle:** Group all temperatures by city
  - **Reduce:** Find max temperature per city
- 

## What You'll Do in the Notebook

### Exercise 2.1: Create Weather Dataset

- Generate sample weather data
- Upload to HDFS from notebook
- Verify upload

### Exercise 2.2: Load Data with Spark

- Initialize `SparkSession`
- Read CSV from HDFS
- Display sample data

### Exercise 2.3: Find Max Temperature

- Use `groupBy()` and `agg()` operations
- Apply Spark's `max()` function
- Display results

### Exercise 2.4: Understand Execution Plan

- Use `.explain()` to see physical plan
- Identify shuffle operations
- Count execution stages

### Exercise 2.5: Experience Caching

- Cache DataFrame in memory
- Run aggregation twice
- Compare execution times
- See Spark's speed advantage over Hadoop

### Exercise 2.6 (Challenge): Multi-Metric Analysis

- Calculate average, min, max, count
- Multiple aggregations in single operation

---

## Questions to Answer (in notebook)

The notebook contains these reflection questions. **Answer each question in a markdown cell in your notebook.**

**Question 7:** In the MapReduce model, what work happens during each phase for this Spark code?

- **Map phase:** What transformation happens to the input data?
- **Shuffle phase:** What data movement is required and why?
- **Reduce phase:** What aggregation occurs?

**Question 8:** Analyze the execution plan - where does shuffle happen?

**Question 9:** What happens the second time you run a cached operation?

---

## Part 3: Reflection & Cleanup

### Reflection Questions

**Answer these in your notebook as markdown cells.** These are deeper questions to think about after completing the hands-on exercises.

**Question 10: HDFS Replication**

- Why does HDFS default to 3x replication in production instead of 2x or 4x?
- In our lab, we used `replication=1` for single-machine setup. Why is this acceptable for learning but not for production?
- What are the trade-offs between different replication levels?
- How does replication relate to fault tolerance?

*Create a markdown cell and write your answer.*

**Question 11: Data Locality**

- How does HDFS enable data locality for MapReduce?
- Why is this critical for performance at petabyte scale?
- What would happen if computation always ran on different machines than data?

Create a markdown cell and write your answer.

### Question 12: MapReduce vs Spark

- When would you choose Hadoop MapReduce over Spark?
- When would you choose Spark over Hadoop?
- What's the fundamental architectural difference?

Create a markdown cell and write your answer.

### Question 13: Shuffle Phase

- Why is shuffle the most expensive phase in MapReduce?
- How does Spark optimize shuffle compared to Hadoop?
- What causes a shuffle operation in Spark?

Create a markdown cell and write your answer.

---

## Cleanup

After completing the lab, run this cell to clean up:

```
# Remove HDFS files
!hdfs dfs -rm -r /user/$(whoami)/week2/

# Remove local files (optional - you may want to keep for reference)
import os
for f in ['sample.txt', 'downloaded_sample.txt',
'large_transactions.csv']:
    if os.path.exists(f):
        os.remove(f)
        print(f"Removed {f}")
```

---

## Submission Requirements

Submit **ONE PDF** exported from your Jupyter notebook.

Your notebook should contain:

### 1. Part 1: HDFS Commands & Answers

- All HDFS commands run with **!** prefix (output captured in notebook)
- Answers to Questions 1-4 in markdown cells
- Questions 5-6 (optional but recommended)



## 2. Part 2: PySpark Code & Answers

- All code cells with output
- Answers to Questions 7-9 in markdown cells

## 3. Part 3: Reflection Answers

- Answers to Questions 10-13 in markdown cells

## 4. Challenge Exercise (optional - extra credit)

- Code for multi-metric analysis
- Output showing avg, min, max, count per city

How to Export Jupyter Notebook to PDF:

### Option A: From JupyterLab

```
File → Save and Export Notebook As... → PDF
```

### Option B: Command Line

```
jupyter nbconvert --to pdf "SP26 CS570 Week 2 – Lab Notebook Hadoop.ipynb"
```

### Option C: Print to PDF

- Open notebook in browser
- File → Print Preview
- Print to PDF from browser

---

## Troubleshooting

Issue: HDFS commands fail

**Solution:**

```
# Start HDFS if not running
start-dfs.sh

# Check NameNode status
hdfs dfsadmin -report

# Verify NameNode is running
jps | grep NameNode
```

---

Issue: "File already exists" error

**Problem:** Trying to upload a file that already exists in HDFS.

**Why:** HDFS is immutable - you cannot modify files in place. You must delete and re-upload.

**Solution (use -f flag to overwrite):**

```
# Force overwrite (easiest)
hdfs dfs -put -f sample.txt /user/${whoami}/week2/
```

**Or delete first:**

```
# Delete old file
hdfs dfs -rm /user/${whoami}/week2/sample.txt

# Upload new version
hdfs dfs -put sample.txt /user/${whoami}/week2/
```

---

Issue: "Cannot connect to NameNode"

**Check HDFS configuration:**

```
cat $HADOOP_HOME/etc/hadoop/core-site.xml
```

Should show something like:

```
<property>
  <name>fs.defaultFS</name>
  <value>hdfs://localhost:9000</value>
</property>
```

---

Issue: PySpark import errors in notebook

**In notebook cell:**

```
import sys
sys.path.append('/path/to/spark/python')
import pyspark
print(pyspark.__version__)
```

**Or install in notebook:**

```
!pip install pyspark==4.1.1
```

---

Issue: Jupyter can't find PySpark

**Set environment variables before launching:**

```
export SPARK_HOME=/path/to/spark
export PYTHONPATH=$SPARK_HOME/python:$PYTHONPATH
jupyter lab
```

---

Issue: HDFS won't start on Windows

**Check if ports are available:**

```
netstat -ano | findstr :9000
netstat -ano | findstr :9870
```

If ports are in use, modify `hdfs-site.xml` to use different ports.

---

## Additional Resources

**Official Documentation:**

- [HDFS Commands Reference](#)
- [PySpark SQL Documentation](#)
- [Spark Programming Guide](#)

**Tutorials:**

- [MapReduce Tutorial \(Apache\)](#)
- [PySpark Tutorial](#)

**Course Resources:**

- Week 1 setup guide (for environment issues)
  - Course Slack channel for questions
  - Office hours: Thursday 4:00-5:00 PM
- 

## Lab Tips

## Time Management:

- **Don't spend too much time on setup issues** - ask for help quickly
- **Part 1 should take ~20 minutes** - if you're stuck after 5 min on one exercise, move on
- **Part 2 requires focused coding** - minimize distractions
- **Save reflection questions for after lab** - think about them during the week

## Working Effectively:

- **Run cells as you go** - don't wait until the end to check outputs
- **Add markdown notes** - document what you observe and learn
- **Experiment beyond the instructions** - try different HDFS commands, modify the Spark code
- **Read error messages carefully** - they often tell you exactly what's wrong

**Remember:** The goal is not just to complete the exercises, but to understand **why** distributed systems work the way they do. Think about:

- How data moves across the network
- Why replication matters for fault tolerance
- Why Spark's in-memory approach is faster than Hadoop's disk-based approach
- How shuffle operations create network bottlenecks

**Work through each step methodically, take notes, and connect what you're doing back to the lecture concepts!**

---

## Getting Help

### During Lab:

- Raise your hand
- Ask neighbors (collaboration encouraged for debugging!)
- Check Slack channel

### After Lab:

- Office hours: Thursday 4:00-5:00 PM
- Email: ragnar.lesch@sfbu.edu
- Course Slack workspace